Self-Supervised and Interpretable Anomaly Detection Using Network Transformers

Daniel L. Marino, *Member, IEEE*, Chathurika S. Wickramasinghe, *Member, IEEE*, Craig Rieger, *Senior Member, IEEE*, and Milos Manic, *Fellow, IEEE*

***Abstract*—Machine learning and deep neural networks (DNNs) have been proposed as a tool to identify anomalies in computer network communications. However, due the obfuscated nature of off-the-shelf machine learning models, their output often does not provide enough information to isolate the source of the anomaly to take corrective measures. In this article, we introduce the network transformer (NeT), a DNN model for anomaly detection that incorporates the graph structure of the communication network in order to improve interpretability. The presented approach has the following advantages: first, enhanced interpretability by incorporating the graph structure of computer networks; second, provides a hierarchical set of features that enables analysis at different levels of granularity; second, self-supervised training that does not require labeled data. The NeT model was evaluated on a set of anomalous scenarios executed in a real industrial control system. The presented approach successfully identified the anomalies, the devices affected, and the specific connections causing the anomalies, providing a data-driven hierarchical approach to analyze the behavior of a cyber network.**

***Index Terms*—Anomaly detection, cybersecurity, interpretable machine learning, self-supervised learning, transformers.**

I. Introduction

CRITICAL infrastructures, such as critical manufacturing, transportation systems, healthcare and public health systems, defense systems, and financial systems, play a vital role in the United States. Failures, delays, or incapacitation of these infrastructures can have debilitating effects on various areas, including national health, public safety and security, stable energy supply, and economic stability [1], [2]. Consequently, building effective and resilient critical infrastructures is a primary concern for many entities, including government, industry, academia, and the research community [3], [4].

The integration of information and communication technologies (ICTs) into industrial control systems (ICSs) has been driven by the desire for increased efficiency and connectivity [5]. ICTs are essential for ensuring the effective operation of critical infrastructure systems. However, their adoption also introduces vulnerabilities that can be exploited by cyber-criminals, jeopardizing security, and resilience [6]. In critical infrastructure environments, sensitive information and data flow continuously through various subsystems, necessitating robust protection against unauthorized and malicious access. Ensuring the security of these networks is crucial to prevent potential threats and maintain the integrity of critical operations.

Network traffic monitoring is fundamental for ensuring the security and reliability of critical infrastructures. This technology is crucial for protecting systems that depend on ICTs. A key aspect of resilient network traffic monitoring involves the timely identification of abnormal behaviors, which enables engineers to take preventative measures and avert potential catastrophic failures [7]. Over the past decade, machine learning (ML) has emerged as a prominent approach for anomaly detection in critical infrastructure environments [7], [8]. By leveraging ML algorithms, organizations can enhance their ability to detect and respond to irregularities in network traffic, thereby strengthening the overall security and operational stability of their critical systems.

This article presents a novel network anomaly detection approach—the network transformer (NeT)—an ML model that uses graph representations to improve the interpretability of anomaly detection in network traffic monitoring. The presented model allows the identification of anomalies in the network at the global level. It also generates a series of hierarchical features (device-level and connection-level) that allow the identification of devices affected by the anomalies and the connections responsible for the anomalies. The interpretable component uses these device-level and device-level features to help with localizing impacted devices and connections during a system attack, providing a better understanding of the detected anomaly at a global level. The approach leverages self-supervised learning to train the model using unlabeled data. A multiprocessing pipeline is presented as a prototype for scalability to large datasets.

Contributions of this article are as follows.

Received 30 August 2024; revised 13 November 2024; accepted 9 January 2025. Date of publication 27 February 2025; date of current version 21 April 2025. This work was supported in part by the Department of Energy through the U.S. DOE Idaho Operations Office under Contract DE-AC07-05ID14517, in part by the Resilient Control and Instrumentation Systems (ReCIS) Program of Idaho National Laboratory, and in part by the Commonwealth Cyber Initiative, an investment in the advancement of cyber R&D, innovation and workforce development. Paper no. TII-24-4368. *(Corresponding author: Daniel L. Marino.)*

Daniel L. Marino, Chathurika S. Wickramasinghe, and Milos Manic are with the Virginia Commonwealth University, Richmond, VA 23284 USA (e-mail: marinodl@vcu.edu; brahmanacsw@vcu.edu; misko@ieee.org).

Craig Rieger, retired, was with the Idaho National Laboratory, Idaho Falls, ID 83415 USA. He resides in Pocatello, ID 83201 USA (e-mail: crieger@ieee.org).

For more information about CCI, visit cyberinitiative.org.

Digital Object Identifier 10.1109/TII.2025.3534443

1) Graph packet dissection method for extracting hierarchical features (global level, device level, and connection level).
2) Self-supervised NeT model for detecting network anomalies at the global level.
3) Interpretable components that use device level and connection level features for providing a better understanding of the detected anomalies.

The rest of this article is organized as follows. Section II presents the background and related work. Section III describes the proposed NeT approach. Section IV presents the experimental evaluation. Finally, Section V concludes this article.

II. Background and Related Work

Anomaly detection involves identifying patterns in data that deviate from the expected behavior of a system [9]. It can be approached in various ways, including out-of-distribution detection, outlier detection, and novelty detection [10], [11]. While many existing ML approaches effectively detect abnormal behaviors in cyber-physical systems [12], [13], they often suffer from a lack of interpretability. These approaches typically only indicate the presence of an anomaly without providing additional information about its source or cause. This limitation is particularly problematic in monitoring applications, where understanding the root of the problem is crucial for effective response and corrective measures. Many off-the-shelf ML models used for anomaly detection are designed as obfuscated, making their internal workings opaque to users [14].

To address the obfuscated nature of ML models, researchers have explored the field of explainable AI (XAI) [15]. XAI aims to make the decision-making processes of ML models more transparent and understandable [16]. Techniques such as local interpretable model-agnostic explanations (LIME) and SHapley Additive exPlanations (SHAP) [17], [18] are commonly used to provide local explanations for individual predictions made by complex models like transformers. In addition, incorporating domain knowledge into the inductive bias of ML algorithms can also enhance interpretability by guiding the model's learning process in a way that aligns with known principles in the specific application domain [19].

Despite these advancements, many approaches still struggle with the challenge of effectively incorporating and utilizing data structure, which can obscure exploratory analysis and limit interpretability. To address this issue, designing methods that not only detect anomalies but also support exploratory analysis is crucial. Preserving the graph structure of the problem can significantly improve interpretability and diagnosis by providing a clearer representation of the relationships between entities [15]. Graph-based approaches offer several advantages over traditional methods, as they enable computation over discrete entities and their relationships, which is essential for understanding complex systems and diagnosing issues effectively. Graph neural networks have emerged as a promising tool in this context, offering a flexible way to embed relational inductive bias into ML models. Recent research highlights their application in learning the structure of relationships in time-series sensor data and identifying anomalous edges in dynamic graphs [20], [21].

In this study, our approach uses transformer neural networks (TNNs) as the backbone for our model. The choice of TNNs is motivated by their advanced capabilities in sequence modeling tasks. TNNs are currently regarded as state-of-the-art ML models for sequence, language, and image processing [22], [23], [24], [25]. They offer significant advantages over traditional neural networks, particularly in terms of efficiency and scalability, due to their ability to process input data in parallel [26]. TNNs excel in capturing complex temporal and spatial dependencies, which enhances their capability to extract features representing normal system behavior, a critical aspect for effective anomaly detection.

This article considers three unsupervised ML model baselines for anomaly detection: one-class support vector machines (OCSVMs), local outlier factors (LOFs), and autoencoders (AEs). These models are chosen due to their widespread use in unsupervised anomaly detection [27]. Our intention is to demonstrate how the presented NeT approach can be effectively integrated with existing anomaly detection techniques, improving performance and interpretability beyond that of the original algorithms. The use of other supervised anomaly detection algorithms, such as variants of SVM, XGBoost, decision trees, and convolutional neural networks [8], [28], which require labeled data, is beyond the scope of this work. The necessity for extensive labeling demands domain expertise, and it is often a time-consuming and expensive task. This is especially relevant in industrial applications of anomaly detection, where anomalous data is scarce and difficult to collect [29], [30], [31]. The scarcity of abnormal labeled data underscores the value of self-supervised and unsupervised methods, which are the focus of our article.

In addition, this study employs t-distributed stochastic neighbor embedding (t-SNE) to visualize deviations between abnormal and normal system behaviors. t-SNE [32] is a dimensionality reduction technique that maps high-dimensional data to a lower dimensional space while preserving local similarities. By converting high-dimensional distances into probabilities, t-SNE excels in capturing nonlinear relationships, surpassing traditional methods such as principal component analysis and multidimensional scaling [33], [34]. Its effectiveness in visualizing neural network embeddings and clustering results has been well-documented, making it a valuable tool in ML applications [35].

III. Interpretable Anomaly Detection: NeT

In order to create an interpretable model for anomaly detection, we design an approach that leverages the graph structure of computer networks. As shown in Fig. 1, the idea is to have an abstract model of the problem that is shared by the human and the ML model. The abstract model in this case is the graph representation of the computer network being monitored. By embedding this abstract model into the ML approach, we are able to leverage our understanding of the system as a graph in order to extract useful information about the anomalies detected in the system.

Fig. 2 shows the overview of the presented NeT approach. The approach consists of the following:

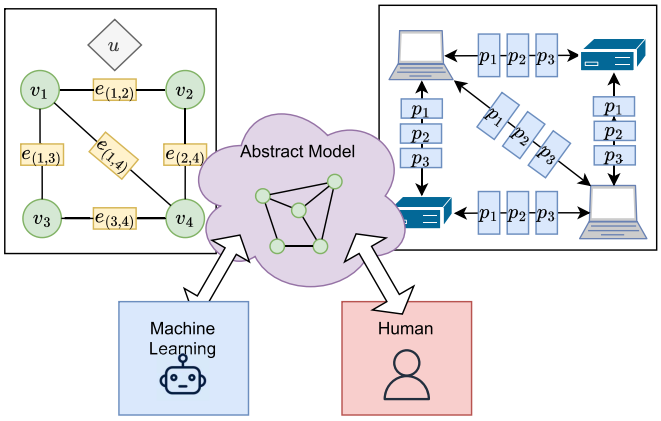

Fig. 1. Graph models as abstract representations for interpretable ML.

1) a packet dissector that groups packages by their respective source and destination addresses;
2) an embedded representation obtained using a TNN;
3) an aggregator that extracts a series of hierarchical network features that represent the communication graph.

The features extracted by the NeT model are fed to anomaly detection algorithms to identify anomalies at different levels of granularity. The approach uses the IP address of the devices in the network to represent the nodes in a graph. Packets communicated between devices represent the edges of the graph. The hierarchical network features are ultimately used for anomaly detection. These hierarchical features include global features representing the entire graph, node features representing each node, and edge features representing individual connections. The approach uses TNNs [26], which are currently the state-of-the-art ML model for sequence modeling tasks. The following sections expand the description of each of the aforementioned components.

A. Graph Packet Dissection

The approach starts by dissecting packet windows in order to identify the source and destination addresses. Packets are grouped by their respective source-destination pairs. A graph is constructed where each node represents an IP address. Edges consist of the list of packets communicated between nodes. Each packet is dissected in order to create an initial feature representation that is suitable for an ML model. We consider two sets of features: transmission control protocol (TCP) features and raw byte features. TCP features are presented in Table I. Raw byte features are presented in Table II.

B. Embedded Packet Representations, Transformer, and Self-Supervised Training

The TNN model is presented in Fig. 3. We use the transformer as it is the current state-of-the-art model for sequence modeling. The transformer uses attention layers that allow capturing long-range dependencies directly. Attention layers also improve the efficiency of the training algorithm as sequences can be processed in parallel; this is in contrast to LSTM and

TABLE I
PACKET FEATURES WHEN USING TCP DISSECTION

Type	Feature	Description
Binary	Direction	Direction of the packet: 0 for i to j, 1 for j to i.
Binary	TCP syn	TCP SYN flag
Binary	TCP ack	TCP acknowledgment flag.
Binary	TCP psh	TCP push function.
Binary	TCP urg	TCP urgent flag.
Categorical	Protocol	Layer of communication: ARP, IP, IPv6, TCP, or UDP.
Categorical	Service	Service (inferred from port used): DNP3, ftp, http, git, telnet, ssh, x11, etc.
Numerical	Len	Size (number of bytes) of the packet.
Numerical	Delta time	Difference in seconds between current packet and previous packet.
Numerical	Source port	Port used by the source device.
Numerical	dest. port	Port used by the destination device.
Numerical	TCP seq	TCP sequence number.
Numerical	TCP ttl	TCP time to live.
Numerical	TCP window	TCP window.
Numerical	Port 22 len	Size of the packet (bytes) when using port 22.
Numerical	Port 23 len	Size of the packet (bytes) when using port 23.

TABLE II
PACKET FEATURES WHEN USING RAW BYTES

Type	Feature	Description
Binary	direction	Direction of the packet: 0 for i to j, 1 for j to i.
Categorical	protocol	Layer of communication: ARP, IP, IPv6, TCP, or UDP.
Numerical	len	Size (number of bytes) of the packet.
Numerical	delta time	Difference in seconds between current packet and previous packet.
Numerical	bytes	List of bytes extracted from the packet (0-512).

GRU models whose inherently sequential nature precludes parallelization [26]. We use a modified version of the transformer model presented in [26], which was originally used to train language models. Like a sentence is composed of a sequence of words, the edges in our network are composed of a sequence of packets. Thus, we use the transformer to encode the list of packets p_k in each edge to a latent representation $z_{(i,j)}$.

The transformer model used in this work is trained using a self-supervised learning approach [36]. The transformer is trained to predict the next n packets (unobserved) given the sequence of past k packets (observed). As shown in Fig. 3, the transformer consists of an encoder–decoder network. The encoder network encodes the input packets into a set of embedded representations that are used by the decoder to predict a series of future packets. This approach allows us to leverage unlabeled data as the input–output packet sequence can be extracted by dividing unlabeled packet windows into two. Once the transformer is trained, we use the encoder network to extract packet features z_n from the edges of the graph Section III-A.

As shown in Fig. 3, the model is composed of the following four types of layers.

1) *Linear*: Applies an affine transformation $f(x) = Wx + b$ of the input using a weight matrix W and a bias vector b.
2) *Feedforward*: Applies an nonlinear transformation using stacked layers $f(x) = \sigma(Wx + b)$ where σ is an activation function. We use the ReLU activation function.
3) *Add and norm*: This layer adds a residual connection followed by layer normalization.
4) *Multihead attention*: This layer consists of several scaled dot-product attention models that evaluate the input in

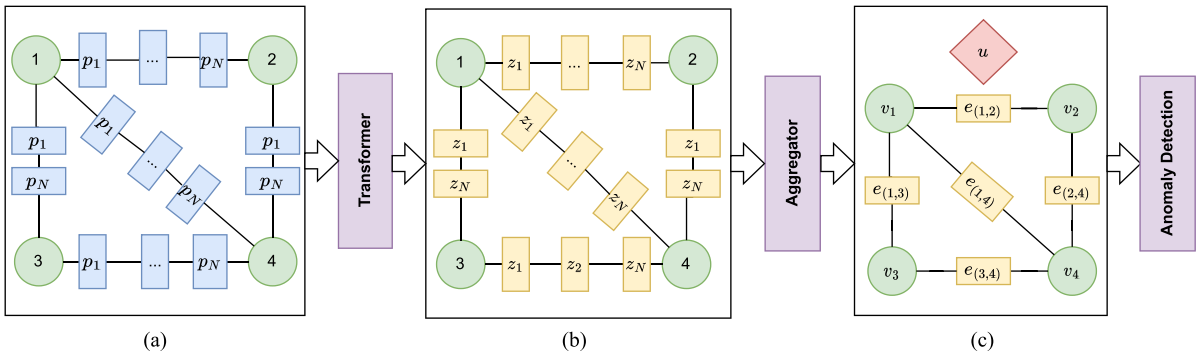

Fig. 2. Overview of the NeT. (a) Graph packet dissection. (b) Embedded representation. (c) Hierarchical network features.

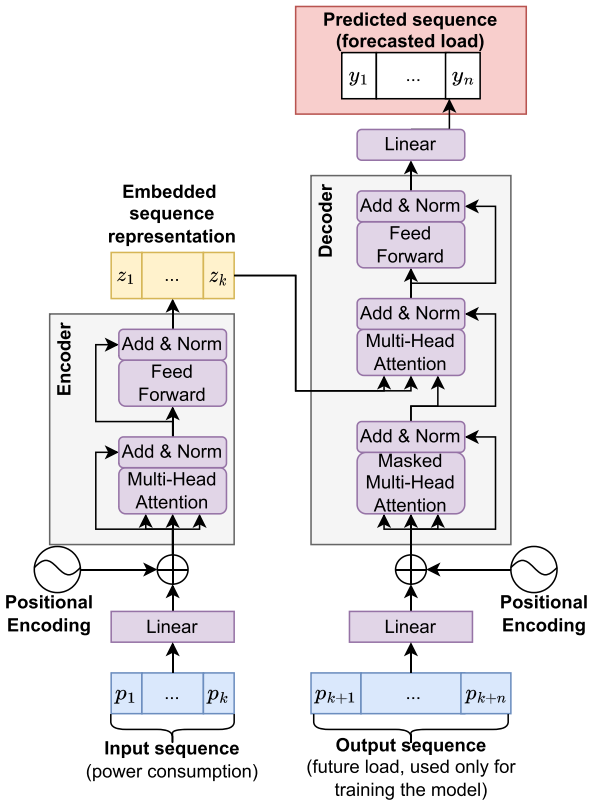

Fig. 3. Transformer model and self-supervised training.

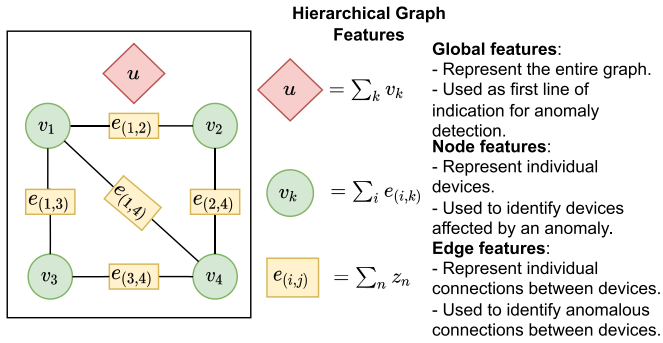

Fig. 4. Hierarchical graph features of the NeT model.

parallel. The scaled dot-product attention is a function that performs an evaluation of a query matrix (Q) over a series of key (K) value (V) matrices

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_z}}\right)V$$

where d_z is the dimension of the encoded feature representations, which is specified when instantiating the transformer model.

Fig. 3 shows $W0$, which is a vector used at the beginning of the target sequence to shift the position of the packets on the right. Shifting the input to the right allows the decoder to predict the next packet given the series of previously predicted symbols, making the transformer model autoregressive [26]. In our implementation, the vector $W0$ is a trainable parameter that is learned when training the transformer model. The position encoding in Fig. 3 uses the sine and cosine functions presented in [26].

The loss function in Fig. 3 is used to evaluate the performance of the transformer network to predict the future window of packets. As shown in Tables I and II, the packets are represented by a series of binary, categorical, and numerical values. We use this distinction to evaluate the loss depending on the type of predicted value. Numerical values use a quadratic loss, while binary and categorical values use a cross-entropy loss.

C. Hierarchical Graph Features

Fig. 4 shows the set of hierarchical graph features of the NeT model. These include global, node, and edge features. These features characterize the computer network at different levels of granularity. NeT hierarchical features are computed on packet windows, which are lists of packets obtained during a specified time window (30 s in this article). To compute NeT features, first each packet feature z_n is encoded using the encoder from the transformer model (see Fig. 3). Edge features $e_{(i,k)}$ are computed by summing the list of encoded packets z_n transferred between nodes i and k. Node features v_k are obtained by summing the edge features $e_{(i,k)}$ attached to the node k. Global features u are obtained by summing all edge features in the graph.

The graph features represent the computer network in a given time window. They represent the network in different layers of abstraction, using a format that is easy to interpret and exploit in order to extract information. We train anomaly detection algorithms for each level of abstraction (global, nodes, edges)

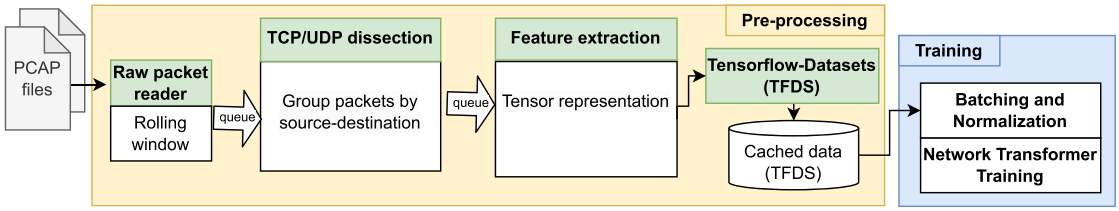

Fig. 5. Data preprocessing pipeline.

in order to detect anomalies and extract information in different levels of granularity. The algorithms are trained using only data collected during the normal operation of the system. In this article, we consider three anomaly detection algorithms: LOF, OCSVM, and AEs.

Global features provide the highest level of abstraction by representing the entire network for a given window of packets. Global features provide a way to identify anomalies effectively without dealing directly with individual nodes or connections. This serves as the first indication of anomalous behavior that considers the network as a whole.

Node features are used to characterize the behavior of individual devices. They provide the next level of abstraction after global features. After an anomaly is detected with the global features, node features are used to identify the devices involved in the anomalous event. Edge features provide the next level of abstraction after node features. Edge features allow us to identify exactly which connections are exhibiting anomalous behavior. Even when we are using a complex DNN transformer model, the hierarchical graph features provide a structure that is easy to understand. This structure can be exploited in order to extract information such as devices and connections involved in an anomaly.

D. Scalability

Network packet data is often characterized by a very high volume of samples. Attacks such as network scans and DOS often flood the computer network with a high volume of packets. As a result, ML algorithms need to be designed in order to handle large quantities of data. Performing the graph packet dissection and training of the NeT required the development of a multiprocessing pipeline in order to scale to these types of datasets.

Fig. 5 shows the developed multiprocessing pipeline used to train the NeT. The pipeline preprocesses a series of packet capture files (PCAP) into a series of features formatted as tensors (multidimensional arrays), ready to be consumed by Tensorflow. The first stage consists of extracting windows of consecutive raw packets using a rolling window. In our case, we used a window of 30 s, but this can be tuned depending on the application. The second processing stage consists of TCP/user datagram protocol (UDP) dissection, which groups packets by the source-destination IP address. The third processing stage extracts features according to Tables I and II. These features are grouped using the source-destination address and then packaged in a tensor representation. These tensor feature

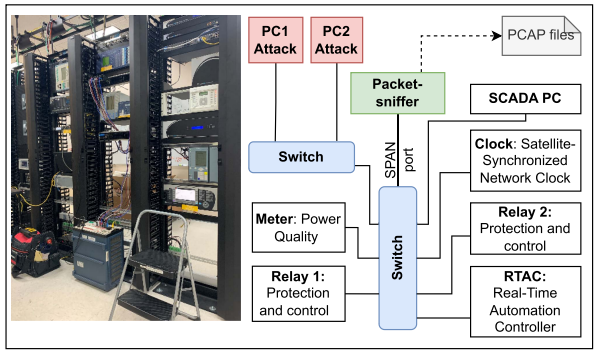

Fig. 6. ICS testbed used to collect data.

representations are then managed by Tensorflow-Datasets, a library that creates a cache of the preprocessed values in order to be consumed efficiently when training the NeT. The training uses stochastic gradient descent, more specifically the adaptive moment estimation (ADAM) algorithm, which scales to very large datasets. Once the NeT is trained, the features can be used for downstream ML operations without retraining the model. The preprocessing pipeline in this work was implemented using Python multiprocessing library, but the same approach can be horizontally scaled with libraries such as Hadoop or Spark.

IV. Experiments

This section presents the experimental analysis of the NeT. We used packet data captured from a real ICS system to evaluate the performance of the presented NeT approach. The performance was evaluated on an anomaly detection task, which included the detection of a set of attack scenarios. We also evaluated the ability of NeT to provide an indication of devices and connections related to a detected anomaly.

A. Data Collection and Processing

The presented approach was evaluated using a dataset of packet captures collected in an ICS. The data were provided by Idaho National Laboratory. The ICS network is presented in Fig. 6. The network is composed of two attack computers, two protection relays, one power quality meter, one real-time automation controller (RTAC), one satellite synchronized network clock, and one SCADA PC. All devices are connected using two network switches. The RTAC is using the DNP3 protocol to communicate with the relays, the meter, and the SCADA PC. A sniffer is connected to a SPAN port in the switch. The SPAN port forwards all the packet data passing through the switch to the

sniffer, which stores the data in PCAP files for later analysis. In total, the collected data consisted of 6.036.046 packets. The data are processed first by performing packet dissection. A rolling window is used to analyze sections of the data, extracting a series of manually engineered features to be used as inputs to ML anomaly detection systems. The normal behavior of the system is learned by ML algorithms such that any behaviors that are different from previously seen data are flagged as anomalies.

For experimental evaluation, the following five types of scenarios were considered.

1) *Normal scenario*: Consisted of normal operation of the system without any cyber disturbance/attack executed.
2) *Flood attack*: Consisted of a flood of packets using hpin3 command with random source IP address. The attack is launched from the PC1 attack device. The attack targets Relays 1 and 2.
3) *Scan attack*: Consisted of a network scan using nmap. The attack was launched from the PC1 attack device. The attack targets Relays 1 and 2. Two attacks were launched. The first attack was an nmap OS fingerprint scan. The second attack was a TCP SYN port scan.
4) *Failed authentication*: Consisted of an unauthorized user who attempted to access remote devices and failed to get access after three attempts. The scenario was launched from PC2 and targeted Relays 1 and 2 through ssh and telnet. After three unsuccessful login attempts, the relays pulsed a series of alarms that were communicated through DNP3.
5) *Setting change*: An unauthorized user that successfully accessed Relays 1 and 2 from PC2 and made changes in the settings of each device. The setting change scenario included changes to the following configurations: current transformer ratio, phase instantaneous overcurrent level, and relay trip equations. When the user logs into the device, the relay pulses an alarm, which is communicated through DNP3 to the RTAC device. After the setting change is executed, the relay pulses another alarm, which is also communicated through DNP3.

For experimental evaluation, the normal scenario data were divided into 80% for training and 20% for testing. Only data from the normal scenario were used to train the models. Thus, training did not require labeling. The remaining 20% of the normal scenario data was used for evaluation, more specifically to measure the false positive ratio. All other four scenarios were considered abnormal. None of the abnormal scenario data was used for training the models. The abnormal scenario data was only used for evaluation, more specifically to measure the anomaly detection ratio. The following sections present the results and analysis of our experiments.

B. Global Level

As described in Section III-C, global features characterize the behavior of the entire network. As mentioned before, LOF, OCSVM, and AEs models were used for comparing the global level performance. These anomaly detection algorithms run on top of the NeT global features. The baseline used the same three anomaly detection algorithms, but it considered a series of hand-engineering statistical features found in the literature [12], [37]. These features were computed across packet windows of 30 s. NeT refers to the NeT using the TCP features from Table I. NeTB refers to the NeT using the raw bytes features from Table II. NeT and NeTB report anomalies detected using packet windows of 30 s as well. The AE model used the reconstruction error to identify anomalies [37]. In this article, we executed attacks at the global level by executing attacks using PC1 and PC2. Thus, the anomaly detection performance was also calculated at the global level.

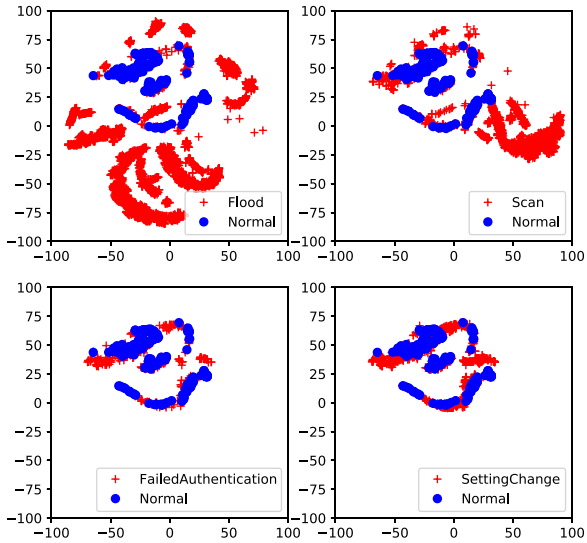

Fig. 7. Global features visualized using the T-SNE algorithm.

The presented global level performance was evaluated using the false positive rate (FPR) and anomaly detection rate (ADR). Results were measured using 5-fold cross-validation, where normal scenario data is split 80% for training and 20% for testing. Data from attack scenarios were only used for performance evaluation. The FPR measures the rate of anomalies for normal scenarios. The ADR measures the rate of anomalies during attack scenarios. FPR is the percentage of incorrectly identified abnormal communication windows given a normal communication duration. ADR is the percentage of correctly identified abnormal communication windows over all the abnormal communication windows. We report ADR measured across all scenarios and for each attack scenario separately. Values of FPR closer to zero and high values of ADR indicate better anomaly detection performance. It is worth clarifying that the ADR may not necessarily be one, as the anomalous scenarios contain a mix of normal and abnormal data [37].

1) Behavior Characterization Analysis: We used NeT global features to characterize the behavior of the entire network, as described in Section III-C. Fig. 7 shows a visualization of the NeT global features. The visualization was obtained with the T-SNE algorithm. The figure shows each anomaly scenario (red) along with data from normal operations (blue). The figure provides a visual reference of the difference between each scenario compared to the normal behavior of the system. We

TABLE III
FPR AND ADR FOR NET MODEL

	FPR train	FPR test	ADR	Flood	Failed Auth	Scan	Setting change
Baseline LOF	0.0931	0.1009	**0.7486**	**0.8855**	**0.3791**	**0.8439**	**0.7063**
Baseline OCSVM	0.0998	0.1062	0.7023	0.8655	0.3059	0.8105	0.6267
Baseline AE	**0.0092**	**0.0363**	0.6923	0.8648	0.2934	0.7915	0.616148
NeT-LOF	0.0896	0.1010	**0.8232**	0.9270	**0.3677**	0.8423	**0.6090**
NeT-OCSVM	0.1005	0.1046	0.5124	0.7954	0.2495	0.1131	0.2732
NeT-AE	**0.0036**	**0.0336**	0.8249	**0.9609**	0.2848	**0.8492**	0.5128
NeTB-LOF	0.0907	0.1019	**0.8471**	0.9514	**0.3465**	**0.8634**	**0.6634**
NeTB-OCSVM	0.1001	0.1001	0.2341	0.2805	0.2495	0.1099	0.2804
NeTB-AE	**0.0097**	**0.0327**	0.8339	**0.9670**	0.1980	0.8539	0.599073

The bold values indicate best performance for the models in the cell.

observed that features from flood and scan scenarios are significantly different from normal behavior. Failed authentication and setting change showed more subtle differences from normal data. This behavior follows our expectations as flood and scan attacks had a larger impact on the network as both introduced a large volume of packets. This visual representation of the global features served as an initial understanding of the data, providing a tool to understand the similarity between different scenarios.

2) Anomaly Detection Analysis: When looking at the results presented in Table III, among baselines, AE provided the lowest FPR while achieving comparable ADR performance. NeTB and NeT provided slightly better FPR with higher ADR when compared with the baseline. We observed a higher ADR for flood and scan scenarios using NeTB and NeT models. However, the baseline provided a slightly better performance on failed authentication and setting change scenarios. NeT-AE and Baseline-AE provided comparable results for failed authentication, with NeTB-AE providing the lowest performance. For setting change, NeTB-AE and baseline AE provided comparable results, with NeT-AE having the lowest performance in this scenario. The baseline and the NT models had specific features indicating activity on ssh and telnet, which helps to explain why they performed better in the failed authentication scenario. On the other hand, setting change was characterized by larger payloads than failed authentication, which helps to explain the better performance of NeTB as it included the raw 512 B of the payload.

Table III shows that the presented NeT and NeTB models provide comparable or better performance in anomaly detection than the baseline. Furthermore, the baseline does not have mechanisms to explore which devices and connections are involved in the anomalies. The following experiments show the capability of NeT to provide an indication of devices affected and anomalous connections using the node and edge features. Given the superior performance of AE compared to LOF and OCSVM, the subsequent sections utilize AE for all experiments.

C. Node Level

Node features were used to characterize the behavior of each device in the network, as described in Section III-C. These features were used to identify the devices affected in each attack scenario. As described in Section IV-A, flood and scan attacks were launched by PC1 while failed authentication and setting change were launched by PC2. Fig. 8 shows the anomalies per-device detected using NeT Node features. The figure shows the number of anomalies per device for each attack scenario.

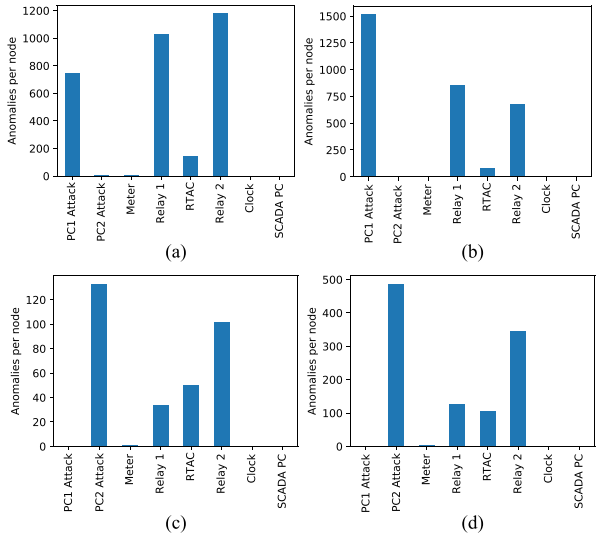

Fig. 8. Anomalies in each device detected with NeT node features. (a) Flood. (b) Scan. (c) Failed Auth. (d) Setting Change.

1) Analysis of Flood Attack: Flood attack shows a high anomaly rate for PC1 [see Fig. 8(a)], which confirms the correct origin of flood attacks.

2) Analysis of Scan Attack: Scan attack also shows a high anomaly rate for PC1 [see Fig. 8(b)], confirming the correct origin.

3) Analysis of Failed Authentication Attack: During failed authentication attacks, PC2 shows a high anomaly rate confirming the origin of the attack [see Fig. 8(c)].

4) Analysis of Settings Change Attack: Setting changes show a high anomaly rate in PC2, confirming the accurate identification of the attack origin [see Fig. 8(d)].

5) Identification of Targeted Devices: A key observation from Fig. 8 is the high anomaly ratio for Relays 1 and 2, which were the devices targeted by the attacker. This highlights the effectiveness of the presented approach in detecting the specific devices under attack.

6) Overall Analysis: The experiments in this section showed that our approach was able to recognize which PC launched

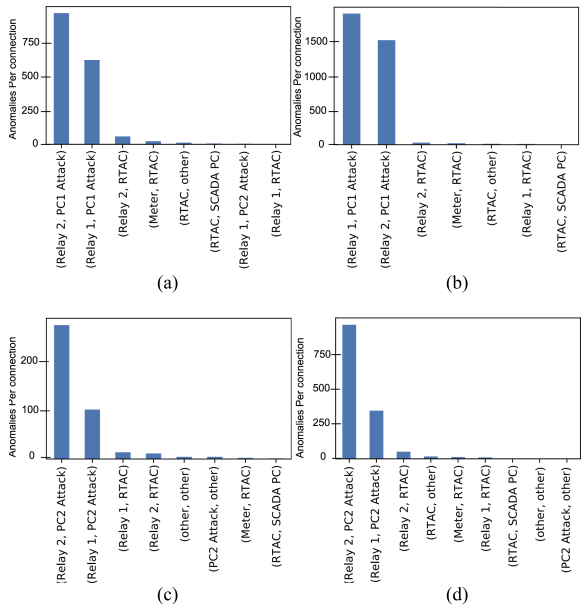

Fig. 9. Anomalous connections detected with NeT edge features. (a) Flood. (b) Scan. (c) Failed Auth. (d) Setting Change.

the attack in each attack scenario. In addition, Fig. 8 shows a high number of anomalies for the RTAC device during failed authentication and setting change. Although the RTAC device was not targeted by the cyber attacks, the relays communicate with the RTAC whenever there is a failed authentication or a setting change. This communication used the DNP3 protocol as described in Section IV-A. A key insight from this observation is that the presented approach not only identified the attacker and target devices but also captured the broader impacts or side effects of the attack, highlighting its ability to detect indirect consequences in addition to direct interactions.

D. Edge Level

We used NeT Edge features to characterize each connection between devices in the network, as described in Section III-C. By running anomaly detection on the edge features, we were able to identify the connections responsible for an anomaly. Fig. 9 shows the results of anomalous connections obtained by analyzing the edge features. The figure shows that the presented approach was able to identify the connections responsible for the anomalies.

1) Analysis on Flood and Scan Attacks: For the flood [see Fig. 9(a)] and scan scenarios [see Fig. 9(b)], we observed that NeT successfully reported the anomalous connections between the attacker (PC1) and the relays.

2) Analysis on Failed Authentication and Setting Change Attacks: For failed authentication [see Fig. 9(c)] and setting change [see Fig. 9(d)], the presented approach was able to identify the anomalous connections, which in this case involved PC2 and both relays.

E. Overall Analysis and Discussion

Figs. 7–9 demonstrate how the proposed approach enables the extraction of information from multiple abstraction layers. This is thanks to the graph structure we have embedded in our methodology. Concretely, our experiments demonstrated the proposed approach has the following main advantages.

1) *Enhance interpretability by incorporating the graph structure of computer networks*: The equivalence between the ML model graph representation and the actual device network communication allows experts to effectively use the model to obtain detailed information, such as the origin of an attack, the devices affected, and the signatures of different attack types. This information is important for users to identify the root causes of anomalies and implement corrective actions. In addition, it provides a shared understanding of the problem between humans and the ML model, which can serve as a bridge to enhance human trust in the ML model outputs.
2) *Provide a hierarchical set of features that enables analysis at different levels of granularity*: The approach integrates multiple layers of abstraction, including global, node, and edge feature representations. This facilitates straightforward backtracking and analysis of model predictions across different levels of detail.
3) *Self-supervised training that does not require labeled data*: Industrial settings produce large volumes of unlabeled data during normal operation. In most cases, it is unpractical, costly, time-consuming, and sometimes impossible to obtain and label abnormal data. Using the presented self-supervised approach, the NeT model can be trained using only data from normal operation, without the need to acquire and label abnormal data.

For the experiments, we chose global-level FPR and ADR metrics to provide a clear and fair comparison across the different anomaly detection algorithms presented. The attacks were executed at a global level only. Therefore, the scope of the experimental setup was to identify attacks at a global level. We did not provide these metrics at a node and connection level because this would require modifying the baseline algorithms we use for comparison, which was outside the scope of the presented work. The purpose of presenting node-level and connection-level anomalous record count in Figs. 8 and 9 was to support our contribution—Enhanced interpretability by incorporating the graph structure of computer networks—demonstrating how NeT edge and node features were used to extract information about devices and connections involved in each anomalous scenario.

V. Conclusion

This article presented the NeT, an ML model that uses graph-based representations to incorporate the structure of a monitored computer network. The presented model uses self-supervised learning in order to train the model without the need for labeled data. NeT provides an approach to extract a series of hierarchical graph features for down-stream ML analysis. In this article, we demonstrated the use of the extracted hierarchical features for anomaly detection. NeT successfully identified anomalies in an ICS network while being able to report devices affected and connections compromised, demonstrating its ability for enhanced interpretability by facilitating the extraction of more detailed information about the anomalies.

The presented approach provided an end-to-end differentiable model for analyzing network packet data, starting from potentially raw byte values up to a global representation of the network graph. Although we used a complex DNN transformer, the hierarchical design of the approach allows to backtrack and analyze the predictions of the model in different levels of granularity. In the experimental evaluation, we demonstrated this analysis starting from a global representation followed by an analysis of individual nodes and connections. The presented analysis demonstrates how the graph structure can be exploited to not only identify anomalies but extract useful information of what devices the anomaly is affecting and which connections are responsible for the anomaly. Because the transformer encoder processes potential raw binary data, the approach can be used as a unified end-to-end methodology for network traffic monitoring that goes from a global view up to individual bytes. Future work will explore the use of existing explainable algorithms such as LIME or SHAP to provide explanations at the packet and possibly byte level.

References

[1] X. Li, C. Xie, Z. Zhao, C. Wang, and H. Yu, "Anomaly detection algorithm of industrial Internet of Things data platform based on deep learning," *IEEE Trans. Green Commun. Netw.*, vol. 8, no. 3, pp. 1037–1048, Sep. 2024.

[2] P. Mishra, A. Vidyarthi, and P. Siano, "Guest editorial: Security and privacy for cloud-assisted Internet of Things (IoT) and smart grid," *IEEE Trans. Ind. Informat.*, vol. 18, no. 7, pp. 4966–4968, Jul. 2022.

[3] A. Tabassum, S. Chinthavali, L. Chen, and A. Prakash, "Data mining critical infrastructure systems: Models and tools," *IEEE Intell. Informat. Bull.*, vol. 19, no. 2, pp. 31–38, Dec. 2018. [Online]. Available: https://www.osti.gov/biblio/1503981

[4] J. Zhu, Y. Yuan, F.-Y. Wang, and G. Wang, "Federated control: A trustable control framework for large-scale cyber-physical systems," *IEEE Trans. Ind. Informat.*, vol. 20, no. 5, pp. 7986–7994, May 2024.

[5] D. L. Marino, C. S. Wickramasinghe, V. K. Singh, J. Gentle, C. Rieger, and M. Manic, "The virtualized cyber-physical testbed for machine learning anomaly detection: A wind powered grid case study," *IEEE Access*, vol. 9, pp. 159475–159494, 2021.

[6] B. Vaagensmith et al., "Review of design elements within power infrastructure cyber-physical test beds as threat analysis environments," *Energies*, vol. 14, no. 5, 2021, Art. no. 1409. [Online]. Available: https://www.mdpi.com/1996-1073/14/5/1409

[7] X. Zhou, Y. Hu, W. Liang, J. Ma, and Q. Jin, "Variational LSTM enhanced anomaly detection for industrial big data," *IEEE Trans. Ind. Informat.*, vol. 17, no. 5, pp. 3469–3477, May 2021.

[8] C. S. Wickramasinghe, D. L. Marino, K. Amarasinghe, and M. Manic, "Generalization of deep learning for cyber-physical system security: A survey," in *Proc. IEEE 44th Annu. Conf. IEEE Ind. Electron. Soc.*, 2018, pp. 745–751.

[9] Y. K. Saheed, O. H. Abdulganiyu, and T. A. Tchakoucht, "A novel hybrid ensemble learning for anomaly detection in industrial sensor networks and SCADA systems for smart city infrastructures," *J. King Saud Univ.-Comput. Inf. Sci.*, vol. 35, no. 5, 2023, Art. no. 101532.

[10] A. Blázquez-García, A. Conde, U. Mori, and J. A. Lozano, "A review on outlier/anomaly detection in time series data," *ACM Comput. Surv.*, vol. 54, no. 3, pp. 1–33, Apr. 2021. [Online]. Available: https://doi.org/10.1145/3444690

[11] X. Du, Z. Fang, I. Diakonikolas, and Y. Li, "How does unlabeled data provably help out-of-distribution detection?," in *Proc. 12th Int. Conf. Learn. Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=jlEjB8MVGa

[12] D. L. Marino et al., "Cyber and physical anomaly detection in smart-grids," in *Proc. Resilience Week (RWS)*, 2019, vol. 1, pp. 187–193.

[13] Z. Zhao, C. Xu, and B. Li, "A LSTM-based anomaly detection model for log analysis," *J. Signal Process. Syst.*, vol. 93, no. 7, pp. 745–751, 2021.

[14] O. Arreche, T. R. Guntur, J. W. Roberts, and M. Abdallah, "E-XAI: Evaluating black-box explainable AI frameworks for network intrusion detection," *IEEE Access*, vol. 12, pp. 23954–23988, 2024.

[15] D. Gunning and D. Aha, "DARPA's explainable artificial intelligence (XAI) program," *AI Mag.*, vol. 40, no. 2, pp. 44–58, 2019.

[16] R. Dwivedi et al., "Explainable AI (XAI): Core ideas, techniques, and solutions," *ACM Comput. Surv.*, vol. 55, no. 9, pp. 1–33, 2023.

[17] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should I trust you? Explaining the predictions of any classifier," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2016, pp. 1135–1144.

[18] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, vol. 30, pp. 4765–4774. [Online]. Available: http://papers.nips.cc/paper/7062-a-unified-approach-to-interpreting-model-predictions.pdf

[19] D. L. Marino and M. Manic, "Physics enhanced data-driven models with variational Gaussian processes," *IEEE Open J. Ind. Electron. Soc.*, vol. 2, pp. 252–265, 2021.

[20] A. Deng and B. Hooi, "Graph neural network-based anomaly detection in multivariate time series," in *Proc. AAAI Conf. Artif. Intell.*, 2021, vol. 35, no. 5, pp. 4027–4035.

[21] L. Cai et al., "Structural temporal graph neural networks for anomaly detection in dynamic graphs," in *Proc. 30th ACM Int. Conf. Inf. Knowl. Manage.*, 2021, pp. 3747–3756.

[22] J. Song, K. Kim, J. Oh, and S. Cho, "Memto: Memory-guided transformer for multivariate time series anomaly detection," *Adv. Neural Inf. Process. Syst.*, vol. 36, pp. 57947–57963, 2024.

[23] J. Lu et al., "Unified-IO 2: Scaling autoregressive multimodal models with vision language audio and action," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2024, pp. 26439–26455.

[24] M. Dehghani et al., "Scaling vision transformers to 22 billion parameters," in *Proc. 40th Int. Conf. Mach. Learn.*, Jul. 2023, pp. 7480–7512. [Online]. Available: https://proceedings.mlr.press/v202/dehghani23a.html

[25] C. Ryali et al., "Hiera: A hierarchical vision transformer without the bells-and-whistles," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 29441–29454.

[26] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, vol. 30, pp. 6000–6010. [Online]. Available: https://proceedings.neurips.cc/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf

[27] P. Bergmann, K. Batzner, M. Fauser, D. Sattlegger, and C. Steger, "The MVTec anomaly detection dataset: A comprehensive real-world dataset for unsupervised anomaly detection," *Int. J. Comput. Vis.*, vol. 129, no. 4, pp. 1038–1059, 2021.

[28] C. Zhou and R. C. Paffenroth, "Anomaly detection with robust deep autoencoders," in *Proc. 23rd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, New York, NY, USA, 2017, pp. 665–674.

[29] Y. An, K. Zhang, Y. Chai, Z. Zhu, and Q. Liu, "Gaussian mixture variational-based transformer domain adaptation fault diagnosis method and its application in bearing fault diagnosis," *IEEE Trans. Ind. Informat.*, vol. 20, no. 1, pp. 615–625, Jan. 2024.

[30] X. Yang, T. Ye, X. Yuan, W. Zhu, X. Mei, and F. Zhou, "A novel data augmentation method based on denoising diffusion probabilistic model for fault diagnosis under imbalanced data," *IEEE Trans. Ind. Informat.*, vol. 20, no. 5, pp. 7820–7831, May 2024.

[31] Y. Wu, Z. Feng, J. Liang, Q. Liu, and D. Sun, "Fault diagnosis algorithm of beam pumping unit based on transfer learning and DenseNet model," *Appl. Sci.*, vol. 12, no. 21, 2022, Art. no. 11091. [Online]. Available: https://www.mdpi.com/2076-3417/12/21/11091

[32] L. Van der Maaten and G. Hinton, "Visualizing data using t-SNE," *J. Mach. Learn. Res.*, vol. 9, no. 11, pp. 2579–2605, 2008.

[33] V. Marx, "Seeing data as t-SNE and UMAP do," *Nature Methods*, vol. 21, pp. 930–933, 2024.

[34] X. Liu, X. Qu, X. Xie, S. Li, Y. Bao, and L. Zhao, "Predicting alloying element yield in converter steelmaking using t-SNE-WOA-LSTM," *Processes*, vol. 12, no. 5, 2024, Art. no. 974. [Online]. Available: https://www.mdpi.com/2227-9717/12/5/974

[35] A. Bibal, V. Delchevalerie, and B. Frénay, "DT-SNE: T-SNE discrete visualizations as decision tree structures," *Neurocomputing*, vol. 529, pp. 101–112, 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0925231223001054

[36] X. Liu et al., "Self-supervised learning: Generative or contrastive," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 1, pp. 857–876, Jan. 2023.

[37] D. L. Marino, C. S. Wickramasinghe, B. Tsouvalas, C. Rieger, and M. Manic, "Data-driven correlation of cyber and physical anomalies for holistic system health monitoring," *IEEE Access*, vol. 9, pp. 163138–163150, 2021.

Daniel L. Marino (Member, IEEE) received the B.Eng. degree in automation engineering from La Salle University, Bogotá, Colombia, in 2015 and the Ph.D. degree in engineering from Virginia Commonwealth University, Richmond, VA, USA, in 2021.

He is currently an Applied Scientist with Amazon.com, Seattle, WA, USA. He has more than eight years of research and development experience, collaborating with U.S. DOE National Labs, universities, and industry. He has authored more than 27 articles in peer reviewed journals and conferences. His research interests include stochastic modeling, deep learning, and explainable AI with applications in cyber-physical systems, cybersecurity, energy, and robotics.

Chathurika S. Wickramasinghe (Member, IEEE) received the B.Sc. degree in computer science from the University of Peradeniya, Peradeniya, Sri Lanka, in 2016 and the Doctoral degree in engineering from Virginia Commonwealth University, Richmond, VA, USA, in 2022.

She is currently a Principle Data Scientist with Capital One, McLean, VA, USA. Her research interests include machine learning, anomaly detection, human behavior segmentation, neural networks, and explainable AI.

Craig Rieger (Senior Member, IEEE) received the B.S. and M.S. degrees in chemical engineering from Montana State University, Bozeman, MT, USA, in 1983 and 1985, respectively, and the Ph.D. degree in engineering and applied science from Idaho State University, Pocatello, ID, USA, in 2008.

He is currently the Managing Director and Consultant with TRECS Consulting. He was the Chief Control Systems Research Engineer and a Directorate Fellow with the Idaho National Laboratory (INL), Idaho Falls, ID, USA, pioneering interdisciplinary research in next-generation resilient control systems. He has also organized and chaired 14 Institute of Electrical and Electronics Engineers (IEEE) technically cosponsored symposia and one National Science Foundation workshop in this new research area. He has authored more than 75 peer-reviewed publications. His doctoral coursework and dissertation focused on measurements and control, with specific application to intelligent, supervisory ventilation controls for critical infrastructure. He has also been a supervisor and technical lead for control-systems engineering groups having design, configuration management and security responsibilities for several INL nuclear facilities and various control-system architectures.

Dr. Rieger is a senior member of IEEE with 20 years of software and hardware-design experience for process control-system upgrades and new installations.

Milos Manic (Fellow, IEEE) received the Dipl.Ing. and M.S. degrees in electrical engineering and computer science from the University of Niš, Niš, Serbia, in 1991 and 1997, respectively, and the Ph.D. degree in computer science fron the University of Idaho, in 2003.

He is currently a Professor with the Computer Science Department, the Director of VCU Cybersecurity Center, Virginia Commonwealth University, Richmond, VA, USA, and the Director of Commonwealth Center for Advanced Computing. He has completed more than 50 research grants in AI/ML in cyber and energy and intelligent controls. He authored more than 200 refereed articles, holds several U.S. patents.

Dr. Manic was the recipient of the 2018 R&D 100 Award for Autonomic Intelligent Cyber Sensor (AICS). He is an inductee of the U.S. National Academy of Inventors (S/M 2023/2019) and a Fellow of Commonwealth Cyber Initiative (specialty in AI & Cybersecurity). He is an IEEE IES President (2024–2025). He was also a recipient of IEEE IES 2019 Anthony J. Hornfeck Service Award, 2012 J. David Irwin Early Career Award, 2017 IEM Best Paper Award, an Associate Editor for IEEE TRANSACTIONS ON INDUSTRIAL INFORMATICS, Open Journal of Industrial Electronics Society, and Senior Life AdCom member. He was Associate Editor for IEEE TRANSACTIONS ON INDUSTRIAL ELECTRONICS, was a founding chair of IEEE Industrial Electronics Society Technical Committee on Resilience and Security in Industry, and was a general chair of IEEE International Conference on Industrial Technology 2023, IEEE International Conference on Human System Interaction 2019, and IEEE IECON 2018.